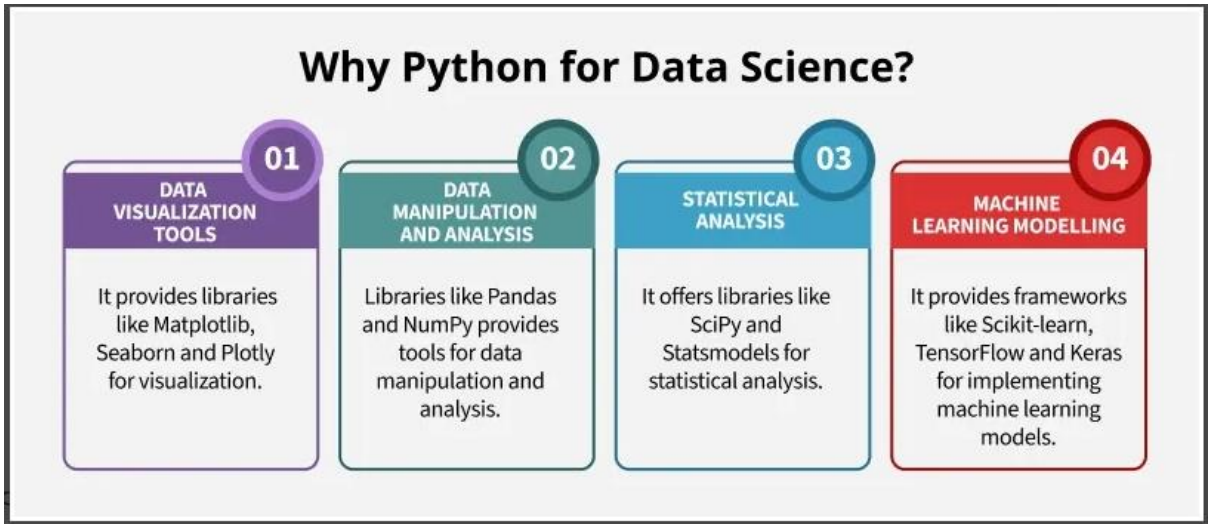
Learn Data Science with Python

- Data Science has become one of the fastest-growing fields in recent years, helping organizations to make informed decisions, solve problems and understand human behavior.
- As the volume of data grows so does the demand for skilled data scientists. The most common languages used for data science are Python and R.



Python Libraries for Data Science

To gain expertise in data science, you need to have a strong foundation in the following libraries:

- Numpy for Numerical Computing
- Pandas for Data Manipulation
- Matplotlib for Data Visulaization
- Seaborn for Data Visualization
- Scikit-learn for Machine Learning

NumPy



What is NumPy?

- NumPy is a Python library used for working with arrays.
- It also has functions for working in domain of linear algebra, fourier transform, and matrices.
- NumPy was created in 2005 by Travis Oliphant. It is an open source project and you can use it freely.
- NumPy stands for Numerical Python.

Why Use NumPy?

- In Python we have lists that serve the purpose of arrays, but they are slow to process.
- NumPy aims to provide an array object that is up to 50x faster than traditional Python lists.
- The array object in NumPy is called ndarray, it provides a lot of supporting functions that make working with ndarray very easy.
- Arrays are very frequently used in data science, where speed and resources are very important.

Why is NumPy Faster than Lists?

- NumPy arrays are stored at one continuous place in memory unlike lists, so processes can access and manipulate them very efficiently.
- This behavior is called locality of reference in computer science.
- This is the main reason why NumPy is faster than lists. Also it is optimized to work with latest CPU architectures.

Installation of NumPy

If you have Python and PIP already installed on a system, then installation of NumPy is very easy.

Install it using this command:

C:\Users\Your Name>pip install numpy

Import NumPy

Once NumPy is installed, import it in your applications by adding the import keyword:

```
import numpy
```

Example:

`import numpy`
 	
`arr = numpy.array([1, 2, 3, 4, 5])`
 	
`print(arr)`
	Output:
	
	[1 2 3 4 5]

NumPy as np

NumPy is usually imported under the np alias.

alias: In Python alias are an alternate name for referring to the same thing.

Create an alias with the as keyword while importing:

```
import numpy as np
```

Example

`import numpy as np`
 	
`arr = np.array([1, 2, 3, 4, 5])`
 	
`print(arr)`
	Output:
	
	[1 2 3 4 5]

Checking NumPy Version

The version string is stored under __version__ attribute.

Example:

`import numpy as np`
 	
`print(np.__version__)`
	Output:
	
	1.16.3

Create a NumPy ndarray Object

- NumPy is used to work with arrays. The array object in NumPy is called ndarray.
- We can create a NumPy ndarray object by using the array() function.

Example:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])

print(arr)

print(type(arr))
```

Output:

```
[1 2 3 4 5]
<class 'numpy.ndarray'>
```

To create an ndarray, we can pass a list, tuple or any array-like object into the array() method, and it will be converted into an ndarray:

Example:

Use a tuple to create a NumPy array:

```
import numpy as np
arr = np.array((1, 2, 3, 4, 5))
print(arr)
```

Output:

```
[1 2 3 4 5]
```

Dimensions in Arrays

- A dimension in arrays is one level of array depth (nested arrays).
- Nested array: are arrays that have arrays as their elements.

1. 0-D Arrays

0-D arrays, or Scalars, are the elements in an array. Each value in an array is a 0-D array.

Example

Create a 0-D array with value 42:

```
import numpy as np

arr = np.array(42)

print(arr)
```

Output:

```
42
```

2. 1-D Arrays

An array that has 0-D arrays as its elements is called uni-dimensional or 1-D array.

These are the most common and basic arrays.

Example:

Create a 1-D array containing the values 1,2,3,4,5:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])

print(arr)
```

Output:

```
[1 2 3 4 5]
```

3. 2-D Arrays

An array that has 1-D arrays as its elements is called a 2-D array.

These are often used to represent matrix or 2nd order tensors.

NumPy has a whole sub module dedicated towards matrix operations called numpy.mat

Example

Create a 2-D array containing two arrays with the values 1,2,3 and 4,5,6:

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

print(arr)
```

Output:

```
[[1 2 3]
 [4 5 6]]
```

4. 3-D arrays

An array that has 2-D arrays (matrices) as its elements is called 3-D array.

These are often used to represent a 3rd order tensor.

Example

Create a 3-D array with two 2-D arrays, both containing two arrays with the values 1,2,3 and 4,5,6:

```
import numpy as np

arr = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5, 6]]])
```

Output:

```
[[[1 2 3]
  [4 5 6]]
```

print(arr)	[[1 2 3] [4 5 6]]]

Check Number of Dimensions?

NumPy Arrays provides the ndim attribute that returns an integer that tells us how many dimensions the array have.

Example

Check how many dimensions the arrays have:

import numpy as np *a = np.array(42)* *b = np.array([1, 2, 3, 4, 5])* *c = np.array([[1, 2, 3], [4, 5, 6]])* *d = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5, 6]]])* *print(a.ndim)* *print(b.ndim)* *print(c.ndim)* *print(d.ndim)*	Output: 0 1 2 3

Higher Dimensional Arrays

An array can have any number of dimensions.

When the array is created, you can define the number of dimensions by using the ndmin argument.

Example

Create an array with 5 dimensions and verify that it has 5 dimensions:

import numpy as np *arr = np.array([1, 2, 3, 4], ndmin=5)* *print(arr)* *print('number of dimensions :', arr.ndim)*	Output: [[[[[1 2 3 4]]]]] number of dimensions : 5

E1. Write a Python program using NumPy to:

1. **Create and print a rank-1 array from a list [1, 2, 3].**
2. **Create and print a rank-2 array from a nested list [[1, 2, 3], [4, 5, 6]].**
3. **Create and print a NumPy array from a tuple (1, 3, 2).**

Code:

```
import numpy as np

# Creating a rank 1 Array

arr = np.array([1, 2, 3])

print(arr)


# Creating a rank 2 Array

arr = np.array([[1, 2, 3],[4, 5, 6]])

print(arr)


# Creating an array from tuple

arr = np.array((1, 3, 2))

print(arr)
```

Output:

```
[1 2 3]


[[1 2 3]

 [4 5 6]]


[1 3 2]
```

E2. Write a Python program using NumPy to demonstrate array slicing and advanced indexing:

1. **Create a 2D NumPy array with the following values:**

   [[-1, 2, 0, 4],
   [4, -0.5, 6, 0],
   [2.6, 0, 7, 8],
   [3, -7, 4, 2.0]]

2. **From this array:**
   - **Extract the first two rows and alternate columns (columns 0 and 2) using slicing.**
   - **Extract and print the elements at the following positions using advanced indexing: (1, 3), (1, 2), (0, 1), (3, 0)**
3. **Print both results with appropriate labels.**

Expected Output (for reference):

first 2 rows and alternate columns(0 and 2):
[[-1 0]
[4 6]]

Elements at indices (1, 3), (1, 2), (0, 1), (3, 0):
[0 6 2 3]

Code:

```
import numpy as np

arr = np.array([[-1, 2, 0, 4],

                [4, -0.5, 6, 0],

                [2.6, 0, 7, 8],

                [3, -7, 4, 2.0]])

# Printing a range of Array

# with the use of slicing method

arr2 = arr[:2, ::2]

print ("first 2 rows and alternate columns(0 and 2):\n", arr2)

# Printing elements at specific Indices

arr3 = arr[[1, 1, 0, 3],

                [3, 2, 1, 0]]
```

print ("\nElements at indices (1, 3), "

```
    "(1, 2), (0, 1), (3, 0):\n", arr3)
```

Output:

first 2 rows and alternate columns(0 and 2):

[[-1. 0.]

[4. 6.]]

Elements at indices (1, 3), (1, 2), (0, 1), (3, 0):

```
[0. 6. 2. 3.]
```

E3. Write a Python program using NumPy to demonstrate basic array operations:

1. **Define two 2x2 NumPy arrays:**
   - **Array a: [[1, 2], [3, 4]]**
   - **Array b: [[4, 3], [2, 1]]**
2. **Perform the following operations:**
   - **Add 1 to every element in array a.**
   - **Subtract 2 from every element in array b.**
   - **Find and display the sum of all elements in array a.**
   - **Perform element-wise addition of arrays a and b.**
3. **Print the result of each operation with a label.**

Code:

```
import numpy as np

# Defining Array 1

a = np.array([[1, 2],[3, 4]])

# Defining Array 2

b = np.array([[4, 3],[2, 1]])


# adding 1 to every element

print ("Adding 1 to every element:", a + 1)

# subtracting 2 from each element

print ("\nSubtracting 2 from each element:", b - 2)

# Sum of array elements

# performing Unary operations

print ("\nSum of all array elements: ", a.sum())


# adding two arrays

# performing Binary operations

print ("\nArray sum:\n", a + b)
```

Output:

Adding 1 to every element:

[[2 3]

 [4 5]]

Subtracting 2 from each element:

[[2 1]

 [0 -1]]

Sum of all array elements: 10

Array sum:

 [[5 5]

 [5 5]]

E4. Write a Python program using NumPy to demonstrate the creation and sorting of structured arrays. Perform the following tasks:

1. **Define a custom NumPy data type dtypes with the fields:**
   - **name (string of max length 10)**
   - **grad_year (integer)**
   - **cgpa (float)**
2. **Create a structured array with the following data:**

   ('Hrithik', 2009, 8.5)
   ('Ajay', 2008, 8.7)
   ('Pankaj', 2008, 7.9)
   ('Aakash', 2009, 9.0)

3. **Sort and display:**
   - **The array in** alphabetical order by name**.**
   - **The array first by** graduation year **and then by** CGPA **(both in ascending order).**

Code:

```
import numpy as np

# set alias names for dtypes

dtypes = [('name', 'S10'), ('grad_year', int), ('cgpa', float)]

# Values to be put in array

values = [('Hrithik', 2009, 8.5), ('Ajay', 2008, 8.7),

          ('Pankaj', 2008, 7.9), ('Aakash', 2009, 9.0)]



# Creating array

arr = np.array(values, dtype = dtypes)

print ("\nArray sorted by names:\n",

           np.sort(arr, order = 'name'))



print ("Array sorted by graduation year and then cgpa:\n",

               np.sort(arr, order = ['grad_year', 'cgpa']))
```

Output:

[(b'Aakash', 2009, 9.) (b'Ajay', 2008, 8.7) (b'Hrithik', 2009, 8.5)

(b'Pankaj', 2008, 7.9)]

Array sorted by graduation year and then cgpa:

[(b'Pankaj', 2008, 7.9) (b'Ajay', 2008, 8.7) (b'Hrithik', 2009, 8.5)

(b'Aakash', 2009, 9.)]

(The b before the string (like b'Aakash') indicates that the string is stored as a byte string (bytes type), not a regular Unicode string (str type))

E5. Write a Python program using NumPy to perform various linear algebra operations on a 3×3 matrix. Follow the steps below:

1. **Create the following 3x3 matrix:**

   A = [[6, 1, 1],
   [4, -2, 5],
   [2, 8, 7]]

2. **Perform the following operations:**
   - **Find and print the rank of matrix A.**
   - **Calculate and display the trace of the matrix.**
   - **Compute and print the determinant of A.**
   - **Find and print the inverse of matrix A.**
   - **Compute and print the matrix raised to the power of 3.**

Note:

The numpy.linalg module provides a set of functions to perform **linear algebra operations** such as:

1. Matrix and Vector Operations

Function	Description
np.linalg.inv()	Computes the **inverse** of a square matrix
np.linalg.det()	Returns the **determinant** of a square matrix
np.linalg.matrix_power(A, n)	Raises matrix A to the power n
np.linalg.matrix_rank()	Computes the **rank** of a matrix
np.trace() *(from NumPy core)*	Computes the **trace** (sum of diagonal elements)

2. Solving Systems of Equations

Function	Description
np.linalg.solve(A, b)	Solves the equation **Ax = b** for x
np.linalg.lstsq(A, b)	Solves least-squares solutions to Ax = b (for non-square matrices)

3. Eigenvalues and Eigenvectors

Function	Description
np.linalg.eig()	Returns **eigenvalues and eigenvectors** of a square matrix
np.linalg.eigvals()	Returns **only eigenvalues** of a matrix

Code:

```
import numpy as np

A = np.array([[6, 1, 1],[4, -2, 5],[2, 8, 7]])

print("Rank of A:",np.linalg.matrix_rank(A))

print("\nTrace of A:", np.trace(A))
```

```
print("\nDeterminant of A:", np.linalg.det(A))

print("\nInverse of A:\n", np.linalg.inv(A))

print("\nMatrix A raised to power 3:\n", np.linalg.matrix_power(A, 3))
```

Output:

```
Rank of A: 3

Trace of A: 11

Determinant of A: -306.0

Inverse of A:

 [[ 0.17647059 -0.00326797 -0.02287582]

 [ 0.05882353 -0.13071895  0.08496732]

 [-0.11764706  0.1503268   0.05228758]]


Matrix A rose to power 3:

 [[336 162 228]

 [406 162 469]

 [698 702 905]]
```